# Capstone Project & Project Guidelines

Participants are **not expected to build enterprise-grade applications**. The focus should be on demonstrating:

- Problem understanding
- Practical use of AI
- Prompt engineering
- Vibe Coding
- API and tool integration
- User experience thinking
- Testing and iteration
- Ability to convert an idea into a working prototype

Each project should result in a **functional Minimum Viable Product (MVP)** that can be demonstrated to evaluators.

---

# PROJECT 1: AI TUTOR

## Project Title

**AI Tutor: Build a Personalized Learning Assistant**

---

## 1. Project Objective

The objective of this project is to build an AI-powered tutoring application that helps students learn a particular subject or topic through personalized explanations, quizzes, feedback, and recommendations.

Instead of creating a general-purpose AI like ChatGPT, participants should create a tutor focused on a **specific learning area**.

Examples:

- Class 8 Mathematics Tutor
- Spoken English Tutor
- Financial Literacy Tutor
- Excel Tutor
- Python Basics Tutor

- Digital Marketing Tutor
- Interview Preparation Tutor
- Geography Tutor
- Banking Exam Preparation Tutor
- Corporate Training Assistant

The tutor should ideally understand what the learner wants to study, explain concepts at an appropriate difficulty level, ask questions, evaluate answers, and suggest what the student should learn next.

The main goal is to demonstrate how AI can make learning more **interactive, adaptive, and personalized**.

---

## 2. Target Audience

Participants should clearly define the intended learner before starting development.

Possible target audiences include:

- School students
- College students
- Working professionals
- Competitive examination aspirants
- Employees undergoing corporate training
- Beginners learning software such as Excel or PowerPoint
- People learning a new language
- Professionals preparing for interviews

The project should explain:

**Who is the learner?**

For example:

“Students aged 13–15 studying Class 8 Mathematics.”

**What problem does the learner face?**

For example:

“Students often understand formulas but struggle to identify which formula should be used for a particular problem.”

**How will the AI Tutor help?**

For example:

“The tutor will explain concepts in simple language, provide worked examples, ask practice questions and recommend additional practice when the student makes mistakes.”

---

## 3. Required Skills

Participants should be able to use:

- Basic Prompt Engineering
- Generative AI tools
- Vibe Coding
- Basic understanding of web applications
- Basic understanding of APIs
- Structured thinking
- User experience design
- Testing and refinement

Advanced programming knowledge should not be required.

Students should rely on AI coding assistants to generate, explain and modify code whenever needed.

---

## 4. Tools and Technologies

Participants may use tools such as:

### AI Model

Any suitable Large Language Model API, such as:

- OpenAI API
- Claude API
- Gemini API

Alternatively, a no-code AI platform may be used.

### Vibe Coding / Development

Possible tools include:

- Replit

- Lovable
- Bolt
- Cursor
- Claude Code
- VS Code with an AI assistant

### Frontend

A simple web interface can be created using:

- HTML
- CSS
- JavaScript

or generated through a Vibe Coding platform.

### Optional Database

If participants want to save learner progress:

- Supabase
- Firebase
- Airtable
- Google Sheets

Database integration should be considered an **optional enhancement rather than a compulsory requirement**.

---

# 5. Step-by-Step Instructions

## Step 1: Select the Learning Topic

Choose one focused learning area.

Avoid extremely broad ideas such as:

“AI Tutor for Everything.”

Instead choose something manageable.

For example:

**AI Tutor for Class 8 Mathematics – Algebra**

Possible topics:

- Linear equations
- Variables
- Algebraic expressions
- Word problems

A narrow scope makes the project easier to build and demonstrate.

---

## Step 2: Define the Tutor's Personality and Teaching Style

Create a system prompt defining how the tutor should behave.

For example:

The tutor should:

- Explain concepts simply
- Avoid unnecessary technical language
- Ask the learner questions
- Give hints before giving answers
- Encourage the learner to try again
- Adjust difficulty based on performance
- Use examples
- Give short quizzes
- Provide constructive feedback

Participants should experiment with their system prompt until the AI behaves consistently.

---

## Step 3: Create the Main Tutor Interface

Build a simple application containing:

- Application title
- Chat window
- User text box
- Send button
- AI response area

The first version can work exactly like a basic chatbot.

Example:

Student:

“What is a linear equation?”

AI Tutor:

“A linear equation is an equation where the highest power of the variable is 1.

For example:

2x + 5 = 15

Would you like me to explain how to solve it?”

---

## Step 4: Add Learning Modes

Participants should add at least **three learning modes**.

Recommended modes:

### Learn

The AI explains the selected concept.

### Practice

The AI generates questions.

### Quiz

The AI asks multiple questions and evaluates the learner.

Optional additional modes:

- Revision
- Ask a Doubt
- Challenge Mode
- Exam Preparation

---

## Step 5: Add Personalization

The learner should provide some basic information before beginning.

For example:

- Name
- Current level
- Topic

- Difficulty preference

Possible difficulty options:

- Beginner
- Intermediate
- Advanced

The AI should modify its explanations accordingly.

For example:

Beginner:

Use simple explanations and examples.

Advanced:

Use challenging problems and fewer hints.

---

## Step 6: Implement Question Generation

The tutor should be able to generate questions dynamically.

Example:

“Give me five beginner-level algebra questions.”

The AI can generate:

1. $x + 4 = 10$
2. $2x = 14$
3. $3x + 2 = 11$
4. $5x – 7 = 18$
5. $4x + 3 = 19$

Questions should not need to be manually stored.

---

## Step 7: Evaluate Learner Answers

The learner should be able to submit an answer.

The AI should respond with:

- Correct / Incorrect
- Explanation

- Correct answer if necessary
- Hint
- Encouragement

Example:

Student:

“x = 5”

AI:

“Almost correct. Let’s check it together.

2x + 5 = 17

Subtract 5:

2x = 12

Divide by 2:

x = 6

Try the next question.”

---

## Step 8: Track Strengths and Weaknesses

Create a simple learner profile.

For example:

**Strong Areas**

- Basic equations
- Addition and subtraction

**Needs Practice**

- Word problems
- Fractions

This information can initially be stored only during the active session.

Persistent storage is optional.

---

### Step 9: Generate a Learning Recommendation

At the end of the session, the AI should generate something such as:

“Today’s Performance”

Questions attempted: 10

Correct: 7

Needs improvement:

- Word problems
- Fractions

Recommended next lesson:

“Linear equations involving fractions”

This provides the personalized learning element of the project.

---

### Step 10: Test the Tutor

Test with different kinds of users.

Try:

- Correct answers
- Incorrect answers
- Completely unrelated questions
- Very easy questions
- Difficult questions
- Spelling mistakes
- Requests for direct answers

Check whether the tutor behaves appropriately.

---

## 6. Expected Outcomes

The final project should ideally include:

- Working AI Tutor web application
- Personalized AI responses

- Learning mode
- Practice mode
- Quiz mode
- Answer evaluation
- Feedback
- Difficulty selection
- Basic learner progress summary
- Deployed online version

Participants should also present a short demonstration.

Suggested demonstration:

1. Choose a learner level.
2. Select a topic.
3. Ask the tutor to explain it.
4. Attempt a practice problem.
5. Give an incorrect answer.
6. Show how AI provides guidance.
7. Complete a short quiz.
8. Show personalized feedback.

---

## 7. Tips and Best Practices

Do not attempt to create an AI tutor for every subject.

Start with one subject and approximately 5–10 topics.

Focus on the learner experience rather than complicated technology.

Ask the AI tutor to guide students instead of immediately giving answers.

Add examples wherever possible.

Use structured prompts.

Test your tutor with both correct and incorrect learner responses.

Keep the user interface simple.

A small tutor that works properly is better than a very ambitious application that does not work reliably.

---

# OPTIONAL ADVANCED FEATURES

Participants who finish early may add:

- Voice interaction
- Student login
- Learning history
- Progress charts
- Gamification
- Badges
- Daily learning streak
- PDF notes upload
- Teacher dashboard
- Automatically generated study plans

These are optional and should not be required for successful completion.

---

---

## RECOMMENDED PROJECT PHILOSOPHY

Participants should follow one simple principle throughout the project:

**Build small → Make it work → Test it → Improve it.**

The goal of these projects is not to create the next ChatGPT, Duolingo, Call of Duty or enterprise call centre.

The goal is to demonstrate that participants can take a real-world idea and use modern AI tools to create a **working digital product**.

A simple product that works reliably is much stronger than a complicated product with many unfinished features.

The ideal final project should make the evaluator think:

**“This may be a prototype, but I can clearly see how this could become a real product.”**

# Project Submission Report template

## Report Structure

| | |
|---|---|
| **1. Cover Page** | Learner Name, Email Id, Mobile Number & Project Title |
| **2. Project Overview** | Brief introduction and objective |
| **3. Problem Statement** | 200–300 words covering the problem, target users, challenges, and proposed AI solution |
| **4. Target Users** | Who will use the solution and what problem they face |
| **5. Proposed Solution** | How the project addresses the identified problem |
| **6. Key Features** | Main features/functionality of the project |
| **7. Solution Architecture** | Flow diagram showing how the solution works |
| **8. Tools & Technologies** | AI models, development tools, APIs, databases, hosting platforms, etc. |
| **9. Working Prototype** | Prototype link + screenshots/demo details |
| **10. Prompts Used** | Important prompts used for development, configuration, content generation, and debugging |
| **11. Testing Report** | Minimum 10 test scenarios with expected result, actual result, and status |
| **12. Challenges & Solutions** | Problems encountered, errors, debugging approach, and improvements |
| **13. Final Outcome** | What was successfully built and key learnings |

| | |
|---|---|
| **14. Future Enhancements** | Optional features/improvements that could be added later |
| **15. Project Demo** | Demo/public URL wherever applicable |

**All the best for your Capstone Project...!**